

MCTS-MAJ: Memory-Augmented LLM-as-a-Judge with Monte Carlo Tree Search

Chapter 3: Design and Methodology

Khush Patel

Spring Submission of the thesis in partial fulfillment
of the requirements for the degree of
Bachelor of Science in Computer Science

under the supervision of Professor Bader Rasheed

Innopolis University

2026

Abstract

This chapter presents the design of MCTS-MAJ, a framework for memory-augmented LLM-as-a-Judge evaluation, and the methodology used to assess it. The research proceeds in two stages. The first stage specifies the *Memory-Assisted Judge (MAJ)*: a typed knowledge graph of past evaluations (Policies, Attempts, Issues, Fixes, Semantic categories), a three-stage retrieval pipeline with asymmetrically tuned similarity thresholds, and an organic learning regime in which the judge’s own verdicts populate memory over time. The second stage introduces *MCTS-MAJ*, which embeds two instances of Monte Carlo Tree Search around the MAJ core: *MCTS-Judge* performs test-time scaling in the reasoning layer through dynamically generated, task-specific evaluation subtasks, and *MCTS-Retrieval* performs agentic search over the memory graph using a set of graph-level tools. To evaluate the framework without the confounds of our previous protocol, the chapter defines a leakage-free evaluation design that partitions the EvalBench dataset by question and freezes memory during testing, together with four memory provenance conditions (no memory, self-written, oracle, and three poisoning rates) that probe a single scientific question: *when does persistent memory help LLM judges, and when does it corrupt evaluation?* The chapter concludes with a discussion of limitations, ethical considerations, and the concrete problems that had to be overcome during implementation.

Contents

Abstract	1
1 Design and Methodology	4
1.1 Purpose of the Chapter	4
1.2 Theoretical Background	4
1.2.1 LLM-as-a-Judge Evaluation	4
1.2.2 Memory-Augmented Retrieval	5
1.2.3 Monte Carlo Tree Search	5
1.3 Research Design	5
1.3.1 Two-Stage Research Trajectory	5
1.3.2 Scientific Question	6
1.3.3 Objectives	6
1.3.4 Experimental Setup	6
1.3.5 Evaluation Metrics	6
1.4 Data Collection Methods	7
1.4.1 Dataset Selection	7
1.4.2 Data Source	7
1.5 Data Gathering Process	7
1.6 Data Preparation Methods	7
1.6.1 Leakage-Free Partition	7
1.6.2 Memory Provenance Conditions	8
1.6.3 Frozen Memory During Evaluation	8
1.7 Core System: Memory-Assisted Judge (MAJ)	8
1.7.1 Graph Schema	9
1.7.2 Three-Stage Retrieval Pipeline	9
1.7.3 Organic Memory Growth	9
1.8 Extension: MCTS-MAJ	10
1.8.1 MCTS-Judge (Reasoning Layer)	10
1.8.2 MCTS-Retrieval (Retrieval Layer)	10
1.8.3 Six Evaluation Modes	11

1.9	Data Analysis Methods	12
1.9.1	Exploratory Data Analysis	12
1.9.2	Result Aggregation	12
1.10	Limitations of the Methods	12
1.10.1	Sample Size	12
1.10.2	Computational Demands	12
1.10.3	Domain Specificity	12
1.10.4	Single-Model Evaluation	13
1.11	Ethical Considerations	13
1.11.1	Data Privacy	13
1.11.2	Transparency and Reproducibility	13
1.12	Problems that Had to Be Overcome	13
1.12.1	Evaluation Leakage	13
1.12.2	Hardcoded MCTS Subtasks	13
1.12.3	Unhashable Dataclass Nodes	13
1.12.4	Free-Text Parsing	14
1.13	Recap of Key Points	14

Chapter 1

Design and Methodology

1.1 Purpose of the Chapter

This chapter specifies the design of the MCTS-MAJ framework and the methodology used to evaluate it. Section 1.2 situates the work in the literature on LLM-as-a-Judge, memory-augmented retrieval, and Monte Carlo Tree Search. Section 1.3 states the scientific question, objectives, experimental setup, and metrics. Sections 1.4–1.6 describe the dataset, the gathering process, and the partition scheme used to eliminate evaluation leakage. Section 1.7 specifies the MAJ core system; Section 1.8 specifies the MCTS-MAJ extension. Section 1.9 describes analysis methods. Sections 1.10–1.12 conclude with limitations, ethics, reproducibility, and implementation problems overcome.

1.2 Theoretical Background

1.2.1 LLM-as-a-Judge Evaluation

The *LLM-as-a-Judge* paradigm uses a language model to evaluate the quality of machine- or human-generated responses against a rubric [7, 14]. Prior work including G-Eval [7] and Prometheus [4] has established that strong LLMs can approach human agreement on open-ended evaluation tasks, yet two systematic weaknesses remain. First, *statelessness*: canonical judges treat each evaluation independently, which precludes learning from prior verdicts and produces inconsistent judgments on semantically related items. Second, *single-pass reasoning*: a single forward pass over the prompt conflates distinct evaluative dimensions (for example, coverage, accuracy, depth) into one decision, which has been shown to hurt precision on multi-aspect rubrics. The framework developed in this thesis addresses both weaknesses simultaneously.

1.2.2 Memory-Augmented Retrieval

Retrieval-Augmented Generation [6] conditions a language model on an external, queryable store at inference time. More recent variants add adaptivity: Active Retrieval [3] modifies the query as reasoning proceeds, and Self-RAG [1] uses the model’s own critique signal to accept or reject retrieved context. These systems still operate on flat document stores. In contrast, the memory in this thesis is a *typed property graph* with distinct node kinds for tasks, responses, errors, and their abstract categories, and explicit edges (SATISFIES, CAUSES, RESOLVES, ABSTRACTS_TO) that enable multi-hop structural queries. Typed structure permits retrieval strategies such as “find the *fix* that resolved the *issue* most similar to the one I am now considering”, which are inexpressible in a flat embedding index.

1.2.3 Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) is a simulation-based anytime search algorithm that incrementally builds a tree of states and estimates the value of each node by averaging the outcomes of rollouts passing through it [5, 11]. Each iteration performs four steps: (i) *selection*, where the tree is traversed from the root using an exploration versus exploitation rule; (ii) *expansion*, where a new node is added; (iii) *simulation*, where a rollout is completed to terminal; and (iv) *backpropagation*, where the reward is propagated along the visited path. The Upper Confidence Bound for Trees (UCT) selection rule trades exploitation of high-mean children against exploration of under-visited ones by combining a child’s estimated value with a term that grows when the child has been visited less often than its siblings. Chain-of-thought prompting [13] showed that decomposition improves LLM reasoning; MCTS generalizes this by searching over *alternative decompositions* rather than committing to a single chain. Recent work has applied MCTS to LLM judges on code correctness [12]; this thesis extends that direction with task-agnostic subtasks and an additional MCTS layer over retrieval.

1.3 Research Design

1.3.1 Two-Stage Research Trajectory

The research was conducted in two stages that build on one another. **Stage 1** produced MAJ and empirically characterized it across seven experiments reported in Chapter 4 (cross-model comparison, seeded memory, retrieval ablation, scaling, cross-domain transfer, cold-start learning curve, unit-test evaluation). Stage 1 established both the conditions under which memory benefits LLM judges and a clear capability threshold: only sufficiently strong models exploit memory. **Stage 2** extended MAJ with MCTS at two

layers, motivated by Stage 1 evidence that retrieval quality and reasoning depth each bound final performance. Stage 2 additionally replaced the original evaluation protocol with a leakage-free design (Section 1.6) after the supervisor flagged a latent leakage risk in the paired-response structure of the benchmark.

1.3.2 Scientific Question

The thesis is framed around a single question:

When does persistent memory help LLM judges, and when does it instead inflate or corrupt evaluation?

The question is motivated by a tension. Memory provides contextual signal that can calibrate a judge across a deployment lifetime. Memory can also propagate errors, leak information across related samples, and reinforce the judge’s own biases. A credible answer requires not only positive evidence that memory helps, but also negative evidence that delimits when it does not, and a quantitative account of failure modes when memory is corrupt.

1.3.3 Objectives

1. Design a memory-augmented judge (MAJ) that stores past evaluations in a structured graph with typed retrieval.
2. Extend MAJ with Monte Carlo Tree Search at both the reasoning layer (multi-perspective evaluation) and the retrieval layer (agentic search over the memory graph).

1.3.4 Experimental Setup

Experiments use the EvalsBench dataset [2] (160 samples, 80 unique questions, paired pass/fail). The underlying judge model is GPT-4o, accessed through the OpenAI API; embeddings are 1536-dimensional vectors from `text-embedding-ada-002` [9]. Memory is persisted in a Neo4j graph database with HNSW-backed vector indexes [8]. Code, prompts, and per-sample result CSVs are version-controlled. All runs fix a random seed (default 42), which is logged with every result file to permit exact replication.

1.3.5 Evaluation Metrics

- **Accuracy:** the proportion of test samples on which the judge’s verdict matches the ground-truth label.

- **Latency:** mean wall-clock time per evaluation, measured end-to-end including retrieval and (where applicable) tree search.
- **Memory utilization:** counts of retrieved contrastive attempts, similar issues, and semantic patterns per evaluation, used to characterize retrieval behavior rather than to drive the verdict.

1.4 Data Collection Methods

1.4.1 Dataset Selection

Evalsbench was selected for three reasons. First, it is a published benchmark with an established vanilla baseline of 84.49 percent for gpt-4o-mini, giving a reference point without requiring an external human study. Second, it *deliberately* pairs each question with both a passing and a failing response, which is a stringent stress test for any memory system: a naive pipeline that updates memory during evaluation can leak signal from one paired response to the other. Third, each sample carries machine-readable grading notes with severity markers, enabling precise rubric-aware prompts without post-hoc rubric construction.

1.4.2 Data Source

The data are read from `data/benchmark_df.csv`: 160 rows across six columns (`topic`, `question`, `grading_notes`, `target` with values `pass` or `fail`, `response`, `notes`). The label distribution is balanced (80 pass, 80 fail); 80 unique questions appear twice, once with a passing response and once with a failing one.

1.5 Data Gathering Process

The dataset is read once as a pandas DataFrame; no scraping, crowdsourcing, or augmentation is performed. For each sample, `grading_notes` becomes the `task` seen by the judge and `response` becomes the `agent_output` to be evaluated.

1.6 Data Preparation Methods

1.6.1 Leakage-Free Partition

An earlier version of the evaluation treated each row as independent and updated memory during testing. Because the dataset pairs each question with two responses of opposite labels, this permits an implicit information channel: evaluating the passing response first

writes positive reasoning into memory that can bias the subsequent fail evaluation of the same question, and vice versa. This is a textbook instance of *train/test contamination via related examples*, and it was flagged during intermediate supervisor review.

To eliminate this channel, the dataset is partitioned *by question identifier*, not by row. The set of 80 unique questions is randomly shuffled with a fixed seed and split evenly into a training set of 40 questions and a held-out test set of 40 questions. The sample partition is induced from the question partition: a sample belongs to train if and only if its question belongs to the training question set. Both the pass and fail versions of any given question therefore lie on the same side of the split, and no question ever appears in both train and test.

1.6.2 Memory Provenance Conditions

Four memory conditions are defined to probe the scientific question:

- **No memory (stateless baseline):** the memory graph is empty.
- **Self-written:** MAJ is run on the 80 training samples and its own verdicts are written to the graph. This models an organic deployment in which the judge bootstraps memory from its own (possibly imperfect) past work.
- **Oracle:** ground-truth labels from the dataset are stored directly as Attempts, bypassing the judge. This upper-bounds what memory provenance can contribute, holding the retrieval pipeline fixed.
- **Poisoned:** oracle memory is used, but a fraction of labels is deterministically flipped at one of three rates (10 percent, 20 percent, or 50 percent) using a fixed seed. This stress-tests the system under controlled corruption.

1.6.3 Frozen Memory During Evaluation

In all conditions memory is read-only during evaluation: no new nodes or edges are written while test samples are judged. This is the second half of the leakage fix: even if the partition is clean, updating memory during testing lets one test sample leak into the next.

1.7 Core System: Memory-Assisted Judge (MAJ)

MAJ is the first stage of the research and the substrate on which MCTS-MAJ builds. MAJ consists of a typed property graph, a three-stage retrieval pipeline, and an organic memory-growth regime.

1.7.1 Graph Schema

Nodes take five types: **Policy** (the task or rubric), **Attempt** (a response with a verdict and reasoning), **Issue** (a concrete problem in an attempt), **Fix** (a resolution for an issue), and **Semantic** (an abstract category grouping related issues). Edges encode four relations: **SATISFIES** from an Attempt to the Policy it addresses, **CAUSES** from an Attempt to any Issue it introduces, **RESOLVES** from a Fix to the Issue it resolves, and **ABSTRACTS_TO** from an Issue to its Semantic category. Every node carries a 1536-dimensional embedding of its text content, indexed for approximate nearest-neighbor search.

1.7.2 Three-Stage Retrieval Pipeline

For each evaluation, MAJ constructs a context in three stages:

1. **Contrastive attempts.** A query embedding retrieves the most similar *passing* and *failing* past attempts. The judge thereby sees both what this kind of response has looked like when correct and when flawed.
2. **Similar issues.** The same query retrieves the most similar Issue nodes, surfacing concrete prior problems likely to repeat.
3. **Semantic patterns.** The Semantic categories attached to the retrieved Issues are aggregated, yielding a small set of high-level failure modes to watch for.

Similarity thresholds are tuned *asymmetrically*: positive examples are accepted at a more permissive cosine similarity threshold than negative examples. The asymmetry reflects an error-cost asymmetry: a spurious positive example mildly biases the judge toward pass, but a spurious negative example can cause the judge to flag an otherwise valid response. Empirically this asymmetry reduced false-positive issue flagging without measurable loss in true-positive recall.

1.7.3 Organic Memory Growth

In deployment, memory is not pre-seeded: MAJ writes its own verdicts, extracted issues, and proposed fixes after each evaluation, and a lightweight classifier assigns each issue to a Semantic category (creating a new one if none exists). Chapter 4 reports how accuracy evolves as memory grows from zero to several hundred examples and quantifies the point of diminishing returns.

1.8 Extension: MCTS-MAJ

Stage 2 extends MAJ with two Monte Carlo Tree Search components: one around *reasoning* (MCTS-Judge), one around *retrieval* (MCTS-Retrieval). Both are designed to be composable: the combined system (“Full MCTS”) chains MCTS-Retrieval into MCTS-Judge.

1.8.1 MCTS-Judge (Reasoning Layer)

MCTS-Judge performs *test-time scaling* for the judge by searching over alternative decompositions of the evaluation task into subtasks. For each sample:

1. **Dynamic subtask generation.** A planning call asks the judge to read the task and response and to emit 5–7 task-specific evaluation perspectives (for example, “Coverage of DCF method”). An earlier prototype with nine *hardcoded* code-oriented subtasks was abandoned after it was found to underperform stateless judging on non-code rubrics; the planning step restores domain relevance.
2. **Selection.** At each internal node, the next subtask is chosen by combining UCT (global, exploration-aware) with an LLM self-assessment (local, task-aware) that scores whether exploring a given child would improve evaluation completeness.
3. **Expansion.** An unvisited subtask becomes a new child of the current node.
4. **Simulation.** The judge evaluates the response from the expanded subtask’s perspective, producing a pass/fail decision for that perspective.
5. **Reward.** A trajectory’s majority vote is compared against an independent lightweight judgment; agreement yields a positive reward, disagreement yields zero. This independent check dampens self-agreement loops.
6. **Backpropagation.** Reward flows up the trajectory, updating visit counts and value estimates.

After `num_rollouts` iterations, the best trajectory (by cumulative reward) is selected, and a final global evaluation aggregates its per-subtask analyses into a single verdict.

1.8.2 MCTS-Retrieval (Retrieval Layer)

MCTS-Retrieval replaces the fixed three-stage pipeline with *agentic search* over the memory graph. The action space is a small set of graph-level tools the agent can invoke:

1. `search_by_attempts` — contrastive vector search over Attempts.

2. `search_by_issues` — vector similarity over Issues.
3. `search_by_semantics` — aggregation over Semantic nodes.
4. `traverse_issue_to_fix` — one-hop traversal from Issues to their resolving Fixes.
5. `traverse_semantic_to_issues` — one-hop traversal from Semantic categories to their member Issues.
6. `traverse_policy_to_attempts` — one-hop traversal from Policies to their Attempts.
7. `traverse_attempts_to_semantics` — two-hop traversal from Attempts through Issues to Semantic categories.

Rollouts compose these tools into trajectories. Each trajectory is scored by a weighted sum of *relevance* (mean similarity of retrieved items), *diversity* (distinct tools used), and *volume* (items retrieved), with weights set on a small development set and held fixed thereafter. UCT-guided selection concentrates subsequent exploration on high-reward tool combinations, so the effective retrieval strategy adapts to the query without any hand-coded pipeline. The tool set is extensible: new graph operations (temporal windowing, cluster-based retrieval, etc.) simply enlarge the action space without any other change to the algorithm.

1.8.3 Six Evaluation Modes

The components compose into six evaluation modes that serve as an ablation ladder:

1. **Stateless** — single-pass judge, no memory, no MCTS.
2. **MAJ** — single-pass judge with three-stage memory retrieval.
3. **MCTS-Judge** — MCTS reasoning, no memory.
4. **MCTS-Judge + Memory** — MCTS reasoning over MAJ memory context.
5. **MCTS-Retrieval + Judge** — MCTS retrieval with single-pass judge.
6. **Full MCTS** — MCTS-Retrieval feeds context into MCTS-Judge.

Each component can be turned on or off independently, and each combination isolates a specific contribution (retrieval alone, reasoning alone, their composition).

1.9 Data Analysis Methods

1.9.1 Exploratory Data Analysis

Exploratory analysis confirmed three structural facts exploited by the methodology: 80 unique questions each appear twice with opposite labels; the label distribution is balanced; topics span 80 categories with no topic appearing more than twice. The pair structure motivated the question-level partition in Section 1.6.

1.9.2 Result Aggregation

Each mode-condition combination writes a per-sample CSV with fields `idx`, `topic`, `expected`, `predicted`, `correct`, `latency_s`, and mode-specific extras (for example, trajectory length for MCTS modes). Accuracy is the mean of `correct`; latency is the mean of `latency_s`. Intermediate traces (MCTS tree snapshots, retrieval trajectories) are stored alongside raw results to support post-hoc analysis and error auditing.

1.10 Limitations of the Methods

1.10.1 Sample Size

All Stage 2 (leakage-free) experiments use 80 test samples. A single flip changes accuracy by 1.25 percentage points, so small differences should be interpreted cautiously. Stage 1 used up to 1000 samples but under the earlier (pre-leakage-fix) protocol. Multi-seed runs with confidence intervals are explicitly scoped as future work.

1.10.2 Computational Demands

MCTS modes cost roughly 10 to 13 times more wall-clock time per sample than single-pass modes (40 to 50 seconds versus 4 seconds). This is a real operational constraint for high-throughput deployment; the framework is best positioned where evaluation quality matters more than per-sample cost.

1.10.3 Domain Specificity

EvalsBench is QA grading. Task families with richer multi-aspect rubrics (for example, agent trajectory evaluation in tau-Bench-style benchmarks [10]) are expected to benefit more from MCTS decomposition. Code evaluation, where the original MCTS-Judge was developed [12], is the natural cross-check; results there are scoped for future work.

1.10.4 Single-Model Evaluation

Stage 2 uses GPT-4o throughout. Stage 1 showed that weaker models (gpt-4o-mini) fail to leverage memory — the capability-threshold finding. Results should not be extrapolated across the threshold.

1.11 Ethical Considerations

1.11.1 Data Privacy

EvalsBench is a public benchmark with no personally identifiable information. No private user data is collected or processed.

1.11.2 Transparency and Reproducibility

Code, prompts, configuration, and per-sample result CSVs are version-controlled. Random seeds are fixed at 42 and logged per run; the leakage-free partition script is deterministic given the seed. Each reported number is reproducible from the stored artifacts.

1.12 Problems that Had to Be Overcome

1.12.1 Evaluation Leakage

The original protocol updated memory during evaluation and treated the 160 rows as i.i.d. Given the paired-response structure of EvalsBench, this admitted a latent information channel from one paired response to the other. The fix was to partition by question and freeze memory during testing (Section 1.6).

1.12.2 Hardcoded MCTS Subtasks

The first MCTS-Judge implementation used nine subtasks imported from a code-evaluation reference [12]. These were poorly matched to QA rubrics and contributed to MCTS-Judge *underperforming* stateless in early runs. Dynamic subtask generation restored domain relevance and recovered a positive contribution from MCTS.

1.12.3 Unhashable Dataclass Nodes

MCTSNode was used as a dictionary key during UCT selection but Python’s `@dataclass` defaults to value-based equality, which broke hashing after state mutation. The fix was `@dataclass(eq=False)`, which preserves identity-based hashing.

1.12.4 Free-Text Parsing

Early MCTS simulations parsed “Decision: Yes/No” from free-text LLM output. Approximately 15–20 percent of responses deviated from the expected format and silently misparsed. All relevant LLM calls were migrated to structured outputs via OpenAI’s `responses.parse` with explicit Pydantic schemas, eliminating silent parse failures.

1.13 Recap of Key Points

This chapter specified MCTS-MAJ as a two-stage design. Stage 1 (MAJ) equips an LLM judge with a typed memory graph and a three-stage retrieval pipeline. Stage 2 (MCTS-MAJ) embeds two MCTS instances around the MAJ core: one searches over reasoning decompositions, the other over graph retrieval trajectories. The evaluation methodology is leakage-free by construction (question-level partition, frozen memory during testing) and instrumented across four memory provenance conditions, which together operationalize the thesis’s scientific question. Chapter 4 reports the experiments and results for both stages.

Bibliography

- [1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. *arXiv preprint arXiv:2310.11511*, 2023.
- [2] EvalsBench Contributors. Evalsbench: A benchmark for llm-as-a-judge evaluation on graded question answering. Internal benchmark dataset, 2024.
- [3] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. *arXiv preprint arXiv:2311.11829*, 2023.
- [4] Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, et al. Prometheus: Inducing fine-grained evaluation capability in language models. *arXiv preprint arXiv:2310.08491*, 2023.
- [5] Levente Kocsis and Csaba Szepesvari. Bandit based monte-carlo planning. *European Conference on Machine Learning (ECML)*, 2006.
- [6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *NeurIPS*, 2020.
- [7] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment. *arXiv preprint arXiv:2303.16634*, 2023.
- [8] Neo4j Inc. Neo4j graph database: Vector search and graph indexes. <https://neo4j.com/docs/>, 2023.
- [9] OpenAI. Openai text embeddings: text-embedding-ada-002. <https://platform.openai.com/docs/guides/embeddings>, 2024.
- [10] Quan Shi, Alexandra Zyttek, Pedram Razavi, Karthik Narasimhan, and Victor Barres. Tau-knowledge: Evaluating conversational agents over unstructured knowledge. *arXiv preprint arXiv:2603.04370*, 2026.

- [11] David Silver, Aja Huang, Chris J. Maddison, et al. Mastering the game of go with deep neural networks and tree search. In *Nature*, volume 529, pages 484–489, 2016.
- [12] Yi Wang, Xin Chen, Ming Zhao, and Peng Liu. Mcts-judge: Test-time scaling in llm-as-a-judge for code correctness evaluation. *arXiv preprint arXiv:2502.12468*, 2025.
- [13] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*, 2022.
- [14] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. In *NeurIPS Datasets and Benchmarks Track*, 2023.